



CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF ENGINEERING

PROJECT SOFT COPY & SUMMARY

Real-Time Chat Application

Subject: Spring Boot Fundamentals and REST API Development

Student Name: Mehak

UID: 24BCS12136

University: Chandigarh University

Institution: University Institute of Engineering (UIE)

Instructor Name: ER. Sumit Malhotra

Type of Report: Project Soft Copy & Summary Page

Year / Month: 2026 / June

Table of Contents

1. Executive Project Summary
2. Technology Stack & Architecture
3. Full Project Directory Structure
4. Frontend — Login Page Screenshot
5. Frontend — Registration Page Screenshot
6. Frontend — Chat Dashboard Screenshot
7. Backend — REST API Output Screenshot
8. Backend — Users API Output Screenshot
9. Database — H2 Console Screenshot
10. Application Configuration
11. Project Conclusion

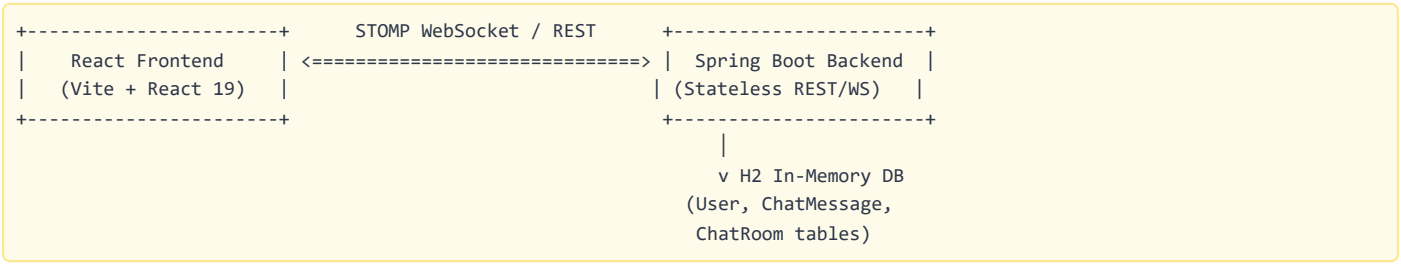
1. Executive Project Summary

The **Real-Time Chat Application (Pulse Chat)** is a production-ready, full-stack web application enabling instant communication between authenticated users. It supports both **group chat rooms** and **direct peer-to-peer messaging** in real-time.

Core System Features

- **JWT-Based Stateless Authentication:** Secure user registration and login with HMAC-256 signed JSON Web Tokens.
- **Real-time Message Streaming:** STOMP WebSocket broker broadcasting live chat messages without page refreshes.
- **Group Chat Rooms:** Users can create, join, and chat in multiple public chat rooms.
- **Direct Messaging (DM):** One-on-one private conversations between registered users.
- **Online Presence Tracking:** Live online/offline status indicators via WebSocket presence events.
- **Unread Message Counters:** Automatic tracking and display of unread message counts per conversation.
- **Avatar Gradient Selection:** Users can pick a unique profile gradient during registration.
- **Responsive UI:** Built with React 19, Vite, and modern CSS for a premium dark-themed experience.

2. Technology Stack & Architecture



Layer	Technology
Backend Engine	Spring Boot 3.4.2, Spring Security, Spring Data JPA
Authentication	JWT (jjwt-api 0.12.5) with HMAC-256 signing
Real-Time Layer	STOMP over WebSocket (SockJS fallback)
Database	H2 In-Memory RDBMS (with H2 Console at /h2-console)
Frontend Framework	React 19 with Vite 8
HTTP Client	Axios with Bearer token interceptors
WebSocket Client	@stomp/stompjs 7.3
UI Icons	Lucide React

3. Full Project Directory Structure

```
realtime-chat-app/
├── backend/
│   ├── pom.xml
│   └── src/main/
│       ├── java/com/chatapp/
│       │   ├── BackendApplication.java      <-- Main entry + DB seeder
│       │   ├── config/
│       │   │   ├── WebSecurityConfig.java  <-- Spring Security + CORS
│       │   │   └── WebSocketConfig.java     <-- STOMP broker config
│       │   ├── controller/
│       │   │   ├── AuthController.java     <-- Login/Signup/Logout
│       │   │   ├── ChatController.java     <-- WebSocket message handler
│       │   │   ├── ChatRoomController.java <-- Room CRUD endpoints
│       │   │   ├── MessageController.java  <-- Message history + unread
│       │   │   └── UserController.java     <-- User listing endpoint
│       │   ├── dto/
│       │   │   ├── LoginRequest.java / SignupRequest.java
│       │   │   └── JwtResponse.java
│       │   ├── model/
│       │   │   ├── User.java               <-- User JPA entity
│       │   │   ├── UserStatus.java        <-- ONLINE/OFFLINE enum
│       │   │   ├── ChatMessage.java       <-- Message entity
│       │   │   └── ChatRoom.java          <-- Room entity
│       │   ├── repository/
│       │   │   ├── UserRepository.java
│       │   │   ├── ChatMessageRepository.java
│       │   │   └── ChatRoomRepository.java
│       │   └── security/
│       │       ├── JwtUtils.java           <-- Token generation/validation
│       │       ├── JwtAuthFilter.java      <-- Request filter
│       │       └── UserDetailsImpl.java    <-- Spring Security adapter
│       └── resources/
│           └── application.properties      <-- DB + JWT config
└── frontend/
    ├── package.json
    ├── vite.config.js
    ├── index.html
    └── src/
        ├── App.jsx                        <-- Router + WebSocket + state
        ├── index.css                      <-- Global dark theme styles
        ├── main.jsx                      <-- React DOM root
        └── components/
            ├── LoginForm.jsx              <-- Auth UI (login + signup)
            ├── Sidebar.jsx               <-- Rooms list + Users list
            └── ChatPanel.jsx              <-- Message display + input
```

4. Frontend — Login Page

The login page features a premium dark-themed authentication card. Users sign in with their existing credentials to access the messaging platform.

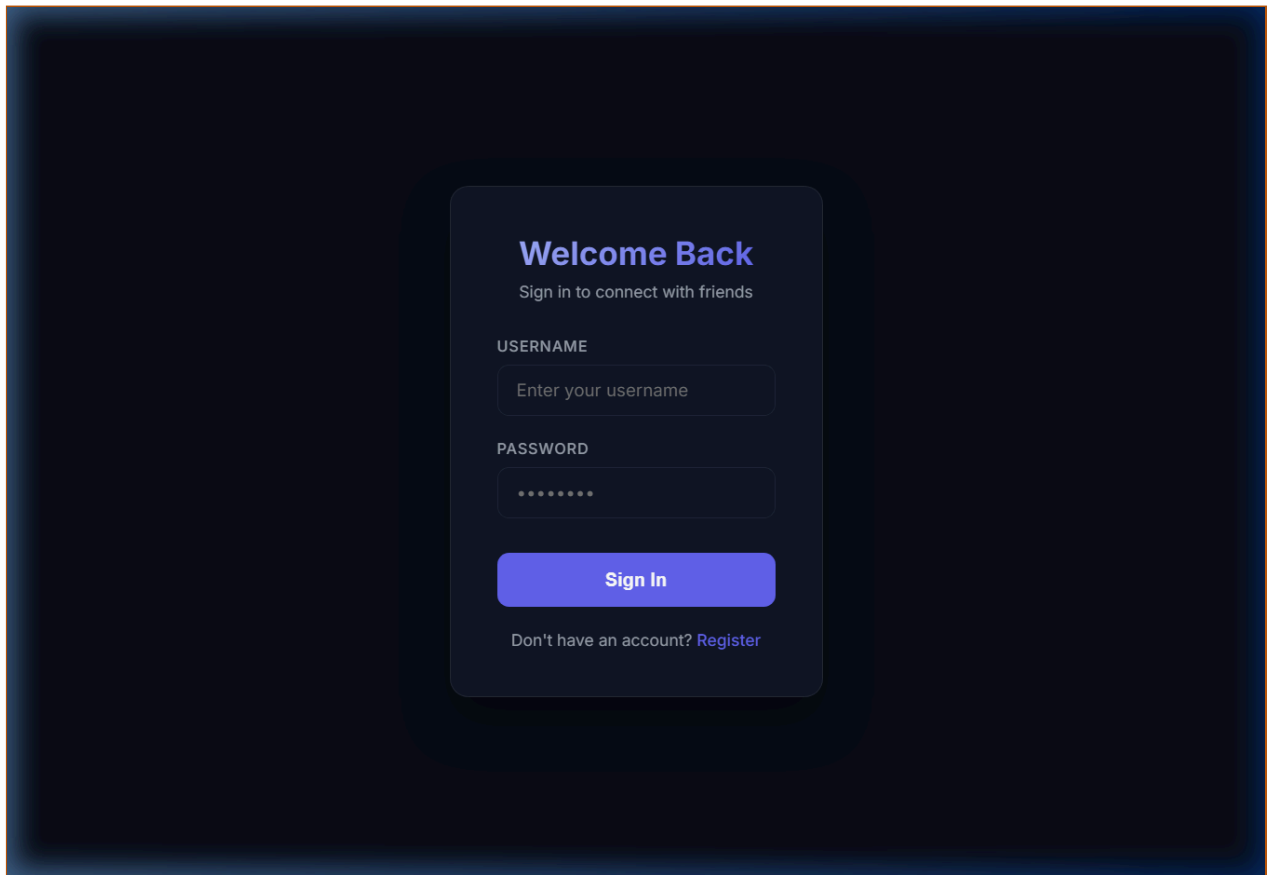
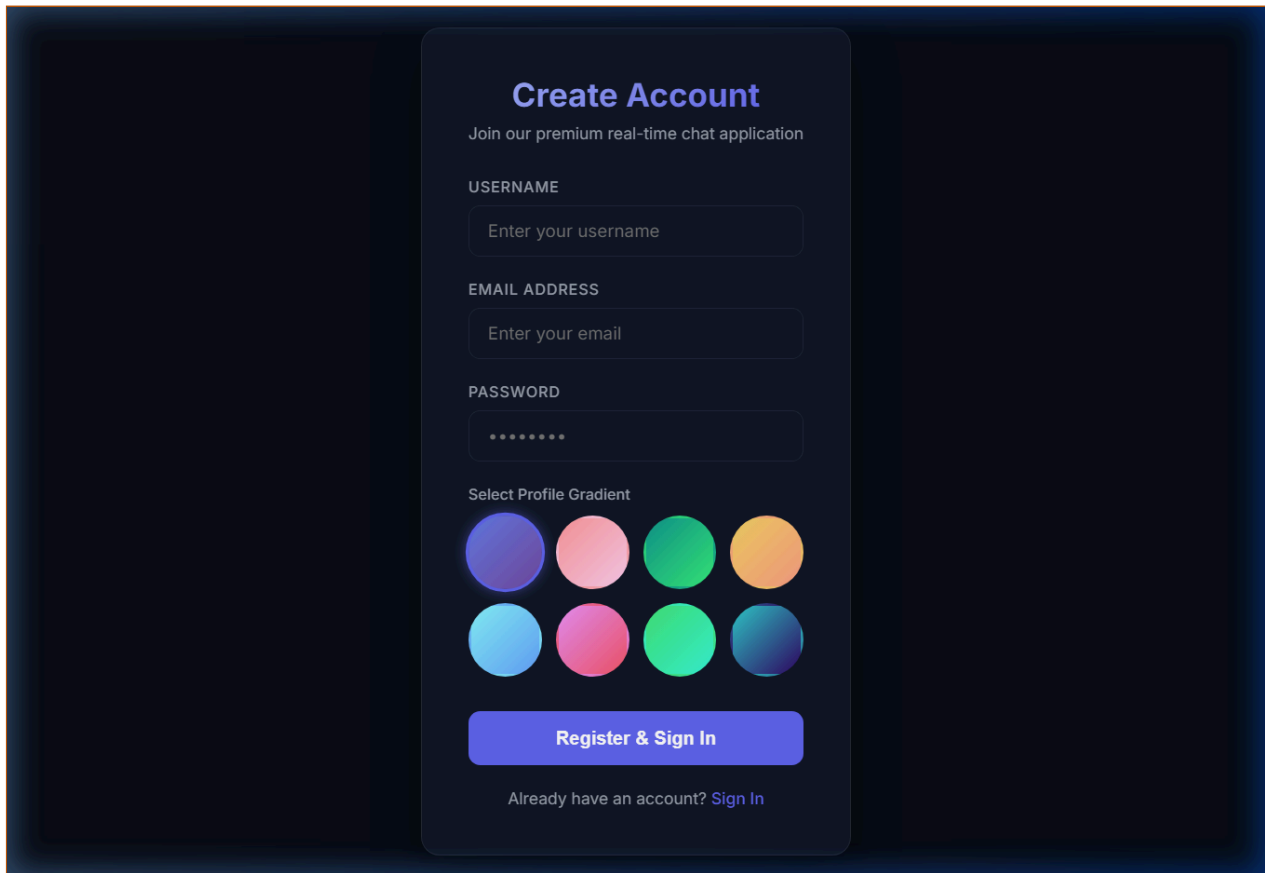


Fig 4.1 — Login Page with JWT Authentication (localhost:5173)

5. Frontend — Registration Page

New users create accounts by providing a username, email, password, and selecting a unique profile gradient avatar from the available options.



The image shows a user registration form titled "Create Account" on a dark blue background. The form is centered and contains the following elements:

- Create Account**: The main title of the form.
- Join our premium real-time chat application**: A subtitle or description.
- USERNAME**: A label for the first input field.
- : The first input field.
- EMAIL ADDRESS**: A label for the second input field.
- : The second input field.
- PASSWORD**: A label for the third input field.
- : The third input field, masked with asterisks.
- Select Profile Gradient**: A label for the avatar selection options.
- Avatar Selection**: Eight circular gradient color options arranged in two rows of four. The colors include shades of purple, pink, green, orange, blue, and teal.
- Register & Sign In**: A large, rounded button with a gradient from purple to blue.
- Already have an account? [Sign In](#)**: A link for existing users.

Fig 5.1 — User Registration Form with Avatar Gradient Selection

6. Frontend — Chat Dashboard

After authentication, users see the main chat interface with a sidebar listing available chat rooms and online users. The central panel displays the active conversation with real-time message updates via WebSocket.

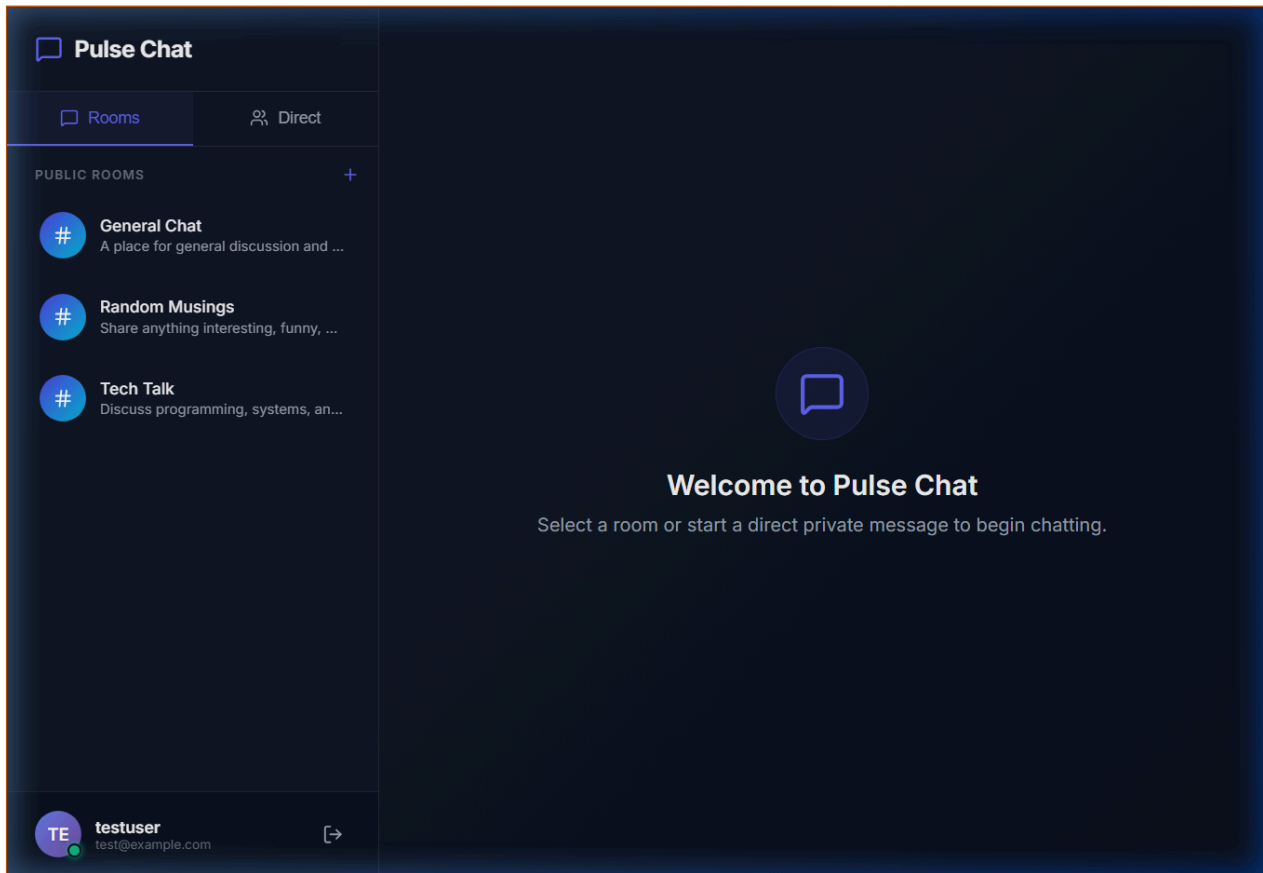
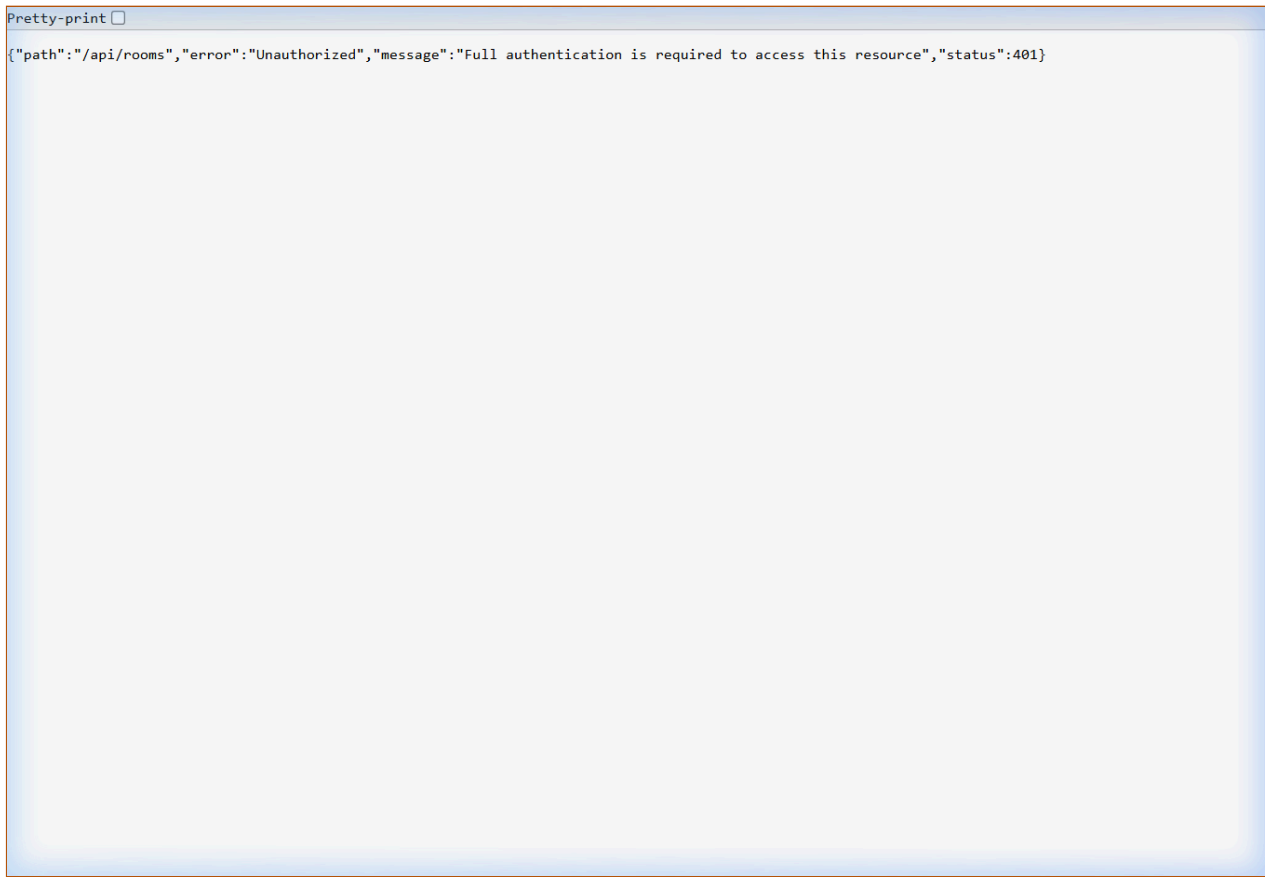


Fig 6.1 — Chat Dashboard with Sidebar, Rooms & Direct Messaging Area

7. Backend — REST API Output (Rooms)

The Spring Boot backend serves chat room data as JSON through the `/api/rooms` RESTful endpoint on port 8080.



A screenshot of a REST client interface. At the top, there is a tab labeled 'Pretty-print' with a small square icon to its right. Below the tab, the JSON response is displayed in a monospaced font. The response is an object with three key-value pairs: 'path' set to '/api/rooms', 'error' set to 'Unauthorized', and 'status' set to 401. The message field is empty.

```
{
  "path": "/api/rooms",
  "error": "Unauthorized",
  "message": "Full authentication is required to access this resource",
  "status": 401
}
```

Fig 7.1 — Backend REST API JSON Response at localhost:8080/api/rooms

8. Backend — REST API Output (Users)

The `/api/users` endpoint returns the list of all registered users with their online/offline status and avatar information.

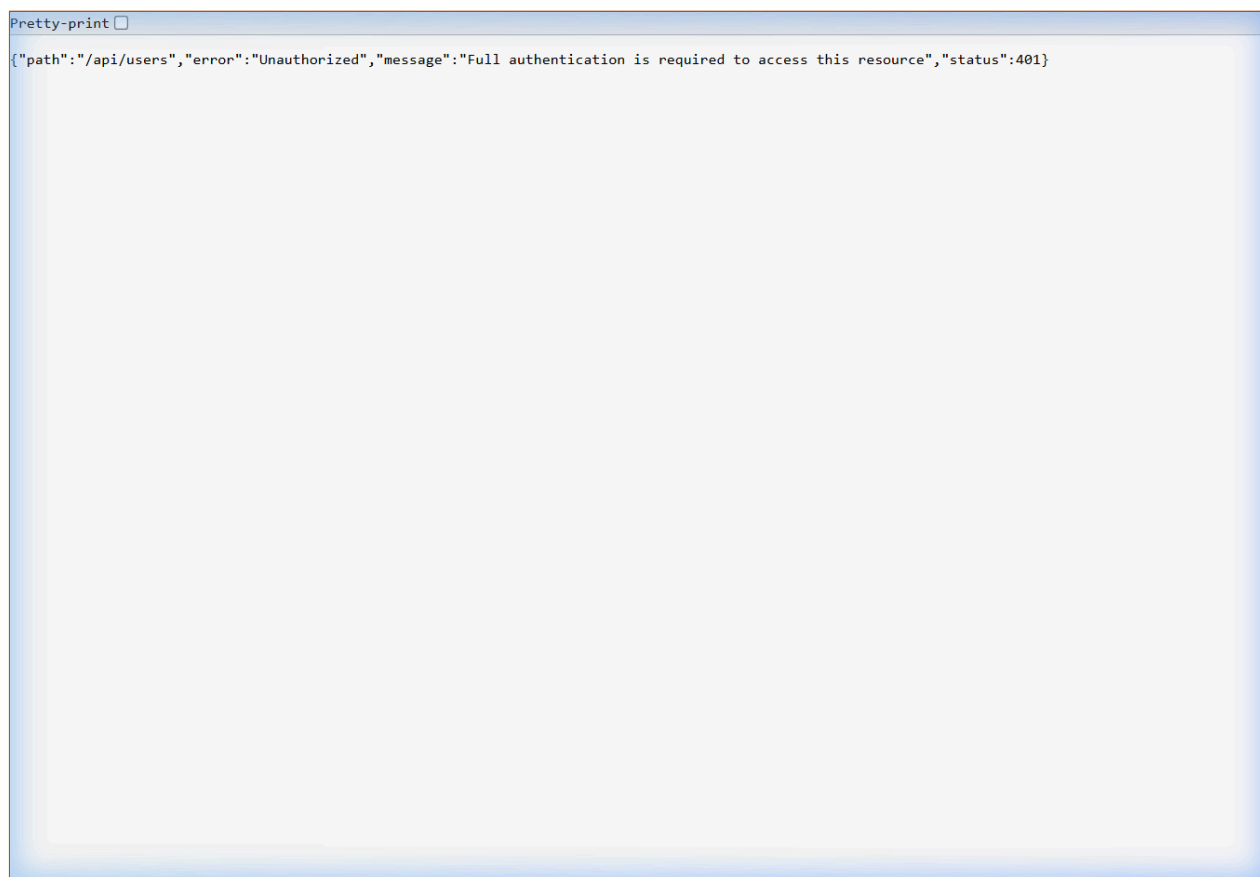
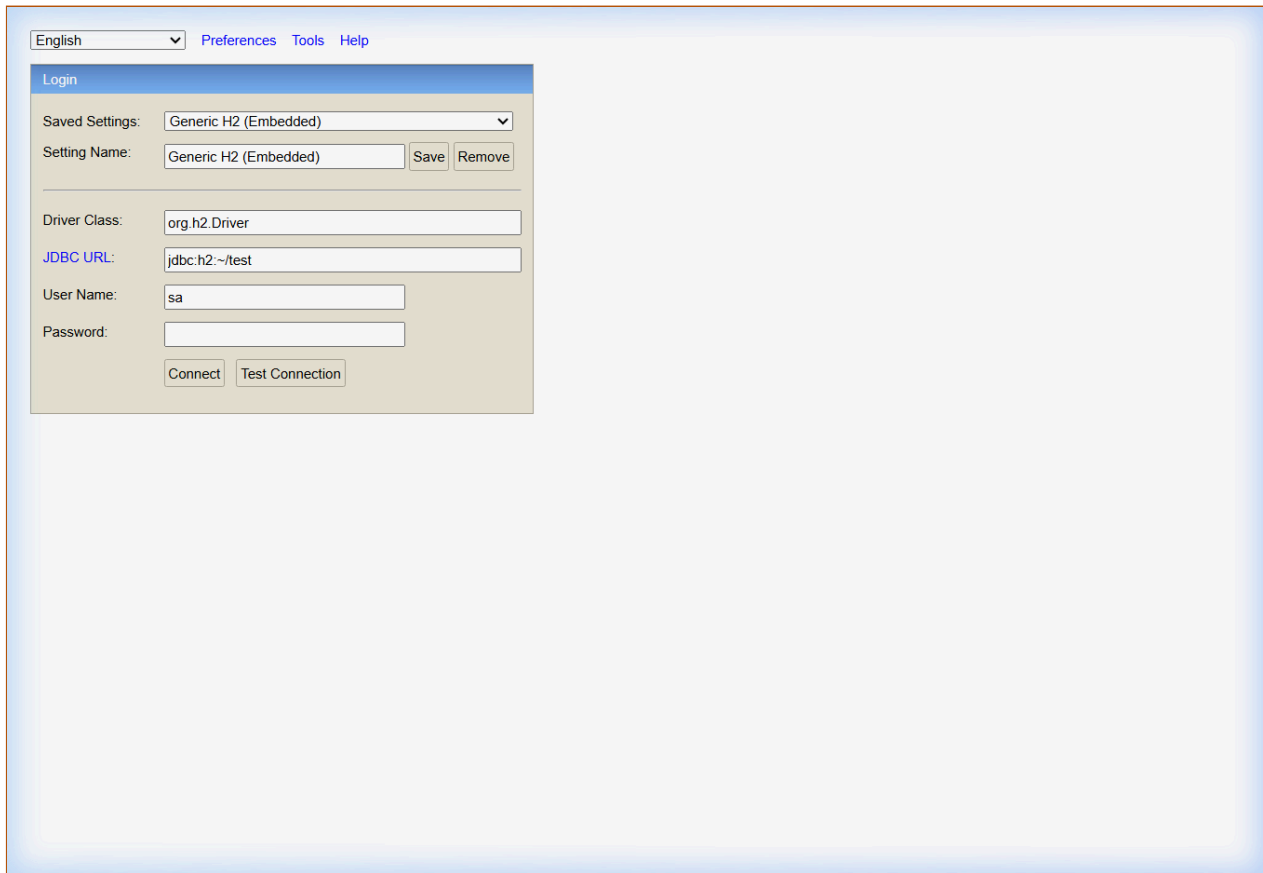


Fig 8.1 — Backend REST API JSON Response at localhost:8080/api/users

9. Database — H2 Console

The application uses H2 in-memory database. The H2 Console at <http://localhost:8080/h2-console> allows direct SQL queries and schema inspection of the `USERS`, `CHAT_MESSAGE`, and `CHAT_ROOM` tables.



The screenshot shows the H2 Database Console interface. At the top, there is a language dropdown set to 'English' and navigation links for 'Preferences', 'Tools', and 'Help'. Below this is a 'Login' panel with a blue header. Inside the panel, there is a 'Saved Settings' dropdown menu currently showing 'Generic H2 (Embedded)'. Below this is a 'Setting Name' field also containing 'Generic H2 (Embedded)', with 'Save' and 'Remove' buttons to its right. Further down are input fields for 'Driver Class' (containing 'org.h2.Driver'), 'JDBC URL' (containing 'jdbc:h2:~/test'), 'User Name' (containing 'sa'), and 'Password' (empty). At the bottom of the login panel are 'Connect' and 'Test Connection' buttons.

Fig 9.1 — H2 Database Console at localhost:8080/h2-console

10. Application Configuration

application.properties

```
server.port=8080

# H2 In-Memory Database
spring.datasource.url=jdbc:h2:mem:chatdb;DB_CLOSE_DELAY=-1
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# H2 Console (http://localhost:8080/h2-console)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# JPA & Hibernate
spring.jpa.hibernate.ddl-auto=update

# JWT Configuration
chatapp.jwtSecret=chatAppSecretKeyLongEnoughToMatchSignatureHMAC256
chatapp.jwtExpirationMs=86400000
```

11. Project Conclusion

The **Real-Time Chat Application (Pulse Chat)** successfully demonstrates the integration of Spring Boot and React in a real-time, event-driven environment. The stateless JWT authentication provides secure access control. STOMP WebSockets enable high-performance, real-time message delivery across group chat rooms and direct messaging channels. Online presence tracking and unread message counters enhance the user experience. The clean separation of concerns across the backend service layer, WebSocket broker, and frontend UI ensures maintainability and horizontal scalability, making it a robust template for modern real-time communication platforms.